오늘 병원,
누구랑 가지?

병원에 가는 길을 함께합니다

TEAM 06

훈희는 못말려

최훈희 · 김신영 · 나희원 · 박현호

PERIOD

2026.10.22 – 10.29

가지?



발표 순서

01 서비스 소개

팀 구성과 서비스 흐름

02 Kotlin 마이그레이션

전환 방법·순서·전후 비교·테스트

03 2차 이후

피드백 반영과 보완

04 상태 흐름과 추가 기능

전체 지도 · 위치·결제·매칭·회원·암호화

05 시연과 상태 다이어그램

실제 화면과 상태 다이어그램

06 인프라와 회고

인프라·성능·남은 과제

01

서비스 소개

팀 구성과 서비스 흐름

팀 구성 및 역할

최훈희 팀장

담당 도메인

회원 · 인증 · 관리자 · 교육

3차 주요 작업

- [x] 개인정보 암호화
- [x] 회원 정지·탈퇴
- [x] 발표 자료 제작

김신영

담당 도메인

공고 · 보고서

3차 주요 작업

- [x] 실시간 위치 공유
- [x] 출발 근접 확인
- [x] Spring AI · Kotlin 전환

나희원

담당 도메인

지원 · 리뷰

3차 주요 작업

- [x] 지원 동시성·상태 정합성
- [x] 동행 종료 위치 삭제
- [x] UI · 시연

박현호

담당 도메인

결제 · 정산 · 이동 · 채팅 · 알림

3차 주요 작업

- [x] DB·배포 환경 이전
- [x] 채팅·알림
- [x] 발표

공고부터 정산까지

01

공고 등록

병원·일정·예상 시간

›

02

선결제

예정 금액 결제

›

03

교육·지원

이수 매니저 지원

›

04

매칭 확정

지원자 한 명 승인

›

05

동행 진행

출발·병원·귀가

›

06

보고서·요약

진료 내용 기록

›

07

완료·정산

실제 시간 금액 조정

02

Kotlin 마이그레이션

전환 방법·순서·전후 비교·테스트

코틀린의 장점

BEFORE Java

AFTER Kotlin

결제 조회

orElseThrow → ?: throw

```java
paymentRepository.findById(paymentId)
  .orElseThrow(() -> new NotFoundException(41, "..."));
```

```kotlin
paymentRepository.findByOrNull(paymentId)
  ?: throw NotFoundException(41, "...")
```

환자 특이사항

null 가능 여부 → 타입에 표시

```java
// null일 수 있는지 타입만 봐서는 모름
private String patientNote;
```

```kotlin
// ? 가 붙어 있으면 쓰는 쪽에서 null 처리 강제
var patientNote: String? = null
```

채팅방 응답 DTO 생성

Stream API → 내장 확장 함수

```java
chatRoom.getChatParts().stream()
  .filter(cp -> !cp.getUser().getId().equals(userId))
    .findFirst()
      .orElseThrow(() -> new NotFoundException("상대방을 찾을 수 없습니다."));
```

```kotlin
chatRoom.chatParts.firstOrNull { it.user.id != userId }
    ?: throw NotFoundException("상대방을 찾을 수 없습니다.")
```

코틀린 마이그레이션 결과

지표	Java 최종 10/23	Kotlin 10/28	비고
Java 파일 수 (main + test)	226 + 53	0	Kotlin 228 + 83파일
Lombok 어노테이션 줄	217	0	—
.builder() 호출 (main + test)	29 + 158	0	—
Optional	58	0	—
null 비교 (== null / != null)	70	20	가독성을 위한 경우만 사용
switch · else if	13 + 12	0 + 1	when 22곳 추가
record → data class	73	78	—
orElseThrow	96	0	—
!! (강제 언래핑)	—	95	요청 데이터 @NotNull을 위한 nullable, 테스트 코드에서 사용

코틀린 마이그레이션 결과

지표	Java 최종 10/23	Kotlin 10/28	비고
Java 파일 수 (main + test)	226 + 53	0	Kotlin 228 + 83파일
Lombok 어노테이션 줄	217	0	—
.builder() 호출 (main + test)	29 + 158	0	—
Optional	58	0	—
null 비교 (== null / != null)	70	20	가독성을 위한 경우만 사용
switch · else if	13 + 12	0 + 1	when 22곳 추가
record → data class	73	78	—
orElseThrow	96	0	—
!! (강제 언래핑)	—	95	요청 데이터 @NotNull을 위한 nullable, 테스트 코드에서 사용

코틀린 마이그레이션 결과

지표	Java 최종 10/23	Kotlin 10/28	비고
Java 파일 수 (main + test)	226 + 53	0	Kotlin 228 + 83파일
Lombok 어노테이션 줄	217	0	—
.builder() 호출 (main + test)	29 + 158	0	—
Optional	58	0	—
null 비교 (== null / != null)	70	20	가독성을 위한 경우만 사용
switch · else if	13 + 12	0 + 1	when 22곳 추가
record → data class	73	78	—
orElseThrow	96	0	—
!! (강제 언래핑)	—	95	요청 데이터 @NotNull을 위한 nullable, 테스트 코드에서 사용

03

2차 이후

피드백 반영과 보완

2차 프로젝트 강사, FT님 피드백과 반영

피드백과 반영	반영 결과
토스 승인 응답의 금액·주문번호·결제키도 대조	반영
결제 기준을 서버로, 클라이언트 값끼리 비교하던 금액을 DB 금액과 대조	반영
의뢰인 프로필은 승인된 동행인만 조회	반영
상태 전이 검증 통합	반영
승인·취소 잠금 순서 통일	반영
매칭 취소 시 지원도 패널티 없이 함께 취소	반영
같은 매니저 중복 일정 승인 충돌을 409로 처리	반영
결제·정산 구현 보완과 테스트 확대	반영

2차 프로젝트 강사, FT님 피드백과 반영

피드백과 반영	반영 결과
토스 승인 응답의 금액·주문번호·결제키도 대조	반영
결제 기준을 서버로, 클라이언트 값끼리 비교하던 금액을 DB 금액과 대조	반영
의뢰인 프로필은 승인된 동행인만 조회	반영
상태 전이 검증 통합	반영
승인·취소 잠금 순서 통일	반영
매칭 취소 시 지원도 패널티 없이 함께 취소	반영
같은 매니저 중복 일정 승인 충돌을 409로 처리	반영
결제·정산 구현 보완과 테스트 확대	반영

결제 검증을 서버로

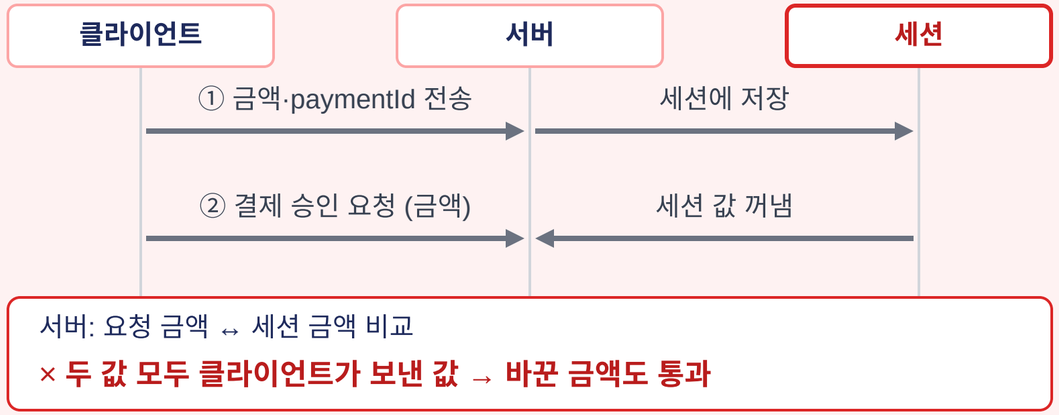
2차의 문제

- × 클라이언트 금액을 그대로 저장
- × 다른 paymentId도 덮어쓰기 가능
- × 바꾼 금액으로 승인될 수 있음

3차의 검증

1. 금액은 서버가 공고 총액으로 만든다
2. 본인 결제인지 확인 — **403-41**
3. DB 금액과 비교 — **400-42**

2차 · 세션에 저장한 클라이언트 값과 비교



3차 · 공고 생성 때 DB에 저장한 값과 비교



승인 전 동행인은 의뢰인 프로필을 볼 수 없다

BEFORE

요청
지원만 한 동행인이
GET /applications/{id}/client-profile

→

서버
지원자 본인인지만 확인
승인 상태는 보지 않음

→

결과
이름·전화번호·비상연락처·
돌봄 메모가 그대로 응답

AFTER

요청
같은 요청
GET /applications/{id}/client-profile

→

서버
본인 확인 + 지원 상태가
ACCEPTED인지 확인

→

결과
403
승인된 동행에서만 조회 가능

```
// ApplicationService.getClientProfile
if (application.escort.id != userId)
    throw ForbiddenException(28, "매칭된 의뢰인의 프로필만 조회할 수 있습니다.")
if (application.status != ApplicationStatus.ACCEPTED)
    throw ForbiddenException(33, "승인된 동행에서만 ...")
```

공고 조회는 회원만

BEFORE

요청
비로그인 사용자가
GET /posts

→

서버
그대로 응답

→

결과
공고 목록이 그대로 응답

AFTER

요청
같은 요청
GET /posts

→

서버
인증 수단(JWT) 확인

→

결과
401
인증된 사용자만 조회 가능

```
// Spring Security Filter
.requestMatchers(HttpMethod.GET, "/api/v1/posts", "/api/v1/posts/**").authenticated()
```

상태 전이 규칙은 enum 한 곳에서 판단한다

문제
매칭 취소된 동행이 출발·완료되고
정산까지 생성됨

원인
허용 상태를 서비스마다
각자 if문으로 검사

```kotlin
// ApplicationStatus.kt
// Application.changeStatus(next) — 엔티티 상태 변경은 여기를 지남 (일괄 거절 UPDATE 1곳 제외)
fun canTransitionTo(next: ApplicationStatus?): Boolean {
    if (next == null) return false
    return when (this) {
        PENDING  -> next == ACCEPTED || next == REJECTED || next == CANCELED
        ACCEPTED -> next == CANCELED || next == NO_SHOW
        REJECTED, CANCELED, NO_SHOW -> false
    }
}
```

canTransitionTo를 가진 enum 6개

PostStatus · ApplicationStatus

PaymentStatus · SettlementStatus

RideStatus · UserStatus

PostStatus 허용 경로

OPEN → MATCHED → IN_PROGRESS → COMPLETED
OPEN → EXPIRED · CANCELED MATCHED → CANCELED · OPEN(노쇼·매니저 정지/탈퇴, 마감 전)
COMPLETED · CANCELED · EXPIRED → 어디로도 못 감

잠금 순서 통일

왜 제어했나? 한 동행인이 일정이 겹치는 두 공고에 지원했는데, 두 의뢰인이 동시에 승인하면?

동행 매니저를 잠그지 않으면

요청 A	공고 A 잠금	→	겹치는 승인 없음	→	승인 완료
요청 B	공고 B 잠금	→	겹치는 승인 없음	→	승인 완료

같은 동행인이 겹치는 일정 2건에 승인 ×

동행 매니저부터 잠그면

요청 A	동행 매니저 잠금	→	공고 → 지원 잠금	→	승인 · 커밋
요청 B	동행 매니저 잠금 대기 …	→	다시 읽기 이미 거절됨	→	409 응답

한 건만 승인, 겹치는 승인은 409 ✓

잠금 순서 통일

승인·취소 모두 User → Post → Application

1단계 동행 매니저 잠금

같은 동행 매니저를 승인하는 요청끼리 한 줄로 처리

2·3단계 공고 → 지원 잠금

잠근 뒤 상태를 다시 읽고 겹치는 일정이면 409

겹치는 대기 지원은 자동 거절

늦게 온 승인은 거절된 지원을 다시 읽고 409-34

동행 매니저부터 잠가서 겹치는 일정은 한 건만 승인

매칭을 취소하면 지원도 함께 취소된다

BEFORE

① 의뢰인 매칭 취소 → ② 공고만 CANCELED
지원은 ACCEPTED 그대로 → ③ 동행인
'출발' 누름 → 통과 → ④ 취소된 공고가
진행 중·정산까지 생성

AFTER

① 의뢰인 매칭 취소
(공고 행 잠금) → ② 대기·승인 지원 모두
패널티 없이 CANCELED → ③ 동행인
'출발' 누름 → 400 → ④ 진행·정산 없음
저장된 위치도 삭제

```
// PostService.matchedCancel
post.matchedCancel()
cancelApplicationsWithoutPenalty(post)        // 대기·승인 지원 모두 취소 + 위치 삭제
// ApplicationService.updateProgress
if (application.status != ApplicationStatus.ACCEPTED) throw InvalidException(29, "승인된 지원만 ...")
```

정산은 성공하고 DB만 실패해도, 두 번 지급하지 않는다

문제

정산 API 호출과 DB 저장이 한 try 안에 있음
→ DB만 실패해도 '정산 실패'로 처리
→ FAILED로 되돌아가 다음 회차에 재정산

해결

① 외부 API와 DB 예외를 따로 처리
② DB 실패 시 FAILED로 되돌리지 않음

```
// SettlementService.requestSettlement (현재)
// row = PENDING/FAILED → PROCESSING 조건부 UPDATE 결과
if (row == 0) throw InvalidException(71, "이미 정산 중이거나 ...")

try {                                          // ① 외부 송금
    response = settlementApi(amount, name, account)
} catch (e: RuntimeException) {
    updateSettlement(id, FAILED)               // 송금 실패만 FAILED
    throw InternalServerErrorException(72, "정산에 실패했습니다.")
}

try {                                          // ② DB 반영
    updateSettlement(id, COMPLETED)
} catch (e: RuntimeException) {
    log.error("정산 DB 변경 실패, 정산은 성공")  // FAILED로 되돌리지 않음
    throw InternalServerErrorException(71, "...")
}
```

그 밖의 보완

항목	무엇을 바꿨나	근거
금액 int → BigDecimal	결제·정산·패널티 금액을 BigDecimal로 · 정산은 반내림 · 응답 DTO는 Long으로	#16 · #63 · PR #102
결제 승인 실패 예외	토스 승인 실패 시 예외를 던져 ‘성공’ 로그가 남지 않음 · DONE 상태 객체 반환	#3 → PR #6
결제 키 우회 프로필 분리	결제 키 우회 로직 프로필 분리(Dev, Test, Prod)	2차 PR #296
관리자 통계 기간	종료일 00시까지만 조회하던 것을 다음 날 00시까지로 · 프론트 우회 코드 제거	#22 → PR #23

04

상태 흐름과 추가 기능

전체 지도 · 위치·결제·매칭·회원·암호화

도메인별 상태값

공고

OPEN	모집 중
MATCHED	매칭 완료
IN_PROGRESS	동행 진행 중
COMPLETED	동행 완료
CANCELED	취소됨
EXPIRED	마감 기한 초과

지원

PENDING	지원 대기
ACCEPTED	지원 승인
REJECTED	지원 거절
CANCELED	지원 취소
NO_SHOW	노쇼

결제

READY	결제 대기
IN_PROGRESS	결제 중
DONE	결제 완료
CANCELED	결제 취소
PARTIAL_CANCELED	결제 부분 취소
DELETED	공고 삭제된 결제

동행 진행

NOT_STARTED	동행 시작 전
DEPARTED	출발
TO_HOSPITAL	병원 이동 중
AT_HOSPITAL	병원 도착
TO_HOME	귀가 중
ARRIVED_HOME	귀가 완료

이동

ACCEPTED	이동수단 선택 완료
IN_PROGRESS	이동 중
COMPLETED	이동 완료

정산

PENDING	정산 전
PROCESSING	정산 중
COMPLETED	정산 완료
FAILED	정산 실패

회원

ACTIVE	활성
SUSPENDED	기간 정지
BANNED	영구 정지

노쇼 패널티

PENDING	적용 전
APPLIED	적용 완료

동행인 등급

SEED	씨앗
SPROUT	새싹
FLOWER	꽃
EGGPLANT	가지

한 건의 동행이 거치는 상태값 ①

정상 흐름 · 예외 · 분기 (실패·노쇼·취소)

① 공고 등록 | ② 결제 승인 | ③ 지원 | ④ 지원 수락 | ⑤ 동행 진행 | ⑥ 동행 완료 | ⑦ 환불·재결제 | ⑧ 정산

공고 Post

모집 중 OPEN

1분 주기

마감 초과 EXPIRED

지원 승인 · 성공 결제 필요

매칭 완료 MATCHED

마감 전: 노쇼·매니저 정지/탈퇴 → 재오픈

모집 중 공고 취소 · 의뢰인 탈퇴/정지

취소됨 CANCELED

의뢰인 취소·탈퇴/정지
마감 후: 노쇼·매니저 정지/탈퇴 → 환불

동행 진행 중 IN_PROGRESS

출발(DEPARTED) 시 전환

귀가 후 의뢰인·관리자 확인

동행 완료 COMPLETED

결제 Payment

결제 대기 READY

결제 중 → 완료 IN_PROGRESS → DONE

승인 실패 → 행 삭제
토스 승인 오류 시 롤백

공고 삭제 → 일괄 취소
DELETED → CANCELED

결제 전액 취소
DONE → 취소

동행 완료 시 (실제 동행 시간 × 시급)과 결제 금액 비교 → 부족하면 추가 결제 생성

추가 결제 생성
READY → 재결제

실제 금액이 더 적으면 차액만 환불

결제 부분 취소
부분 환불

지원 Application

매칭 취소 · 동행인 탈퇴/정지 (패널티 없음)

지원 취소 CANCELED

대기 중 본인 취소

지원 대기 PENDING

의뢰인 거절 ·
다른 지원 승인 / 일정 겹침 시 자동

지원 거절 REJECTED

지원 승인 ACCEPTED

동행 무산 → 노쇼 NO_SHOW

매칭 후 동행인이 취소 (출발 전)
→ 노쇼 패널티 생성 · 등급 재계산 (다음 장)

동행 진행 EscortProgress

시작 전 NOT_STARTED → **출발** DEPARTED → **병원 이동** TO_HOSPITAL → **병원 도착** AT_HOSPITAL → **귀가 중** TO_HOME → **귀가 완료** ARRIVED_HOME

버튼 · 의뢰인 100m 이내
공고 IN_PROGRESS 전환

모든 단계 버튼 · 순서대로

위치 정보 즉시 삭제

이동수단 Ride

이동수단 선택
ACCEPTED ×2 (병원·귀가)

병원행 · 병원 이동(TO_HOSPITAL) 시작 시 이동 중

병원행 이동 IN_PROGRESS → **병원행 완료** COMPLETED → **귀가행 이동** IN_PROGRESS → **귀가행 완료** COMPLETED

귀가행 · 귀가 중(TO_HOME) 시작 시 이동 중

한 건의 동행이 거치는 상태값 ②

정상 흐름 · 예외 · 분기 (실패·정지)

보고서 Report

보고서 작성 조건
본인이 수행한 동행 건만
지원 1건당 1회 · 중복 작성 불가
공고·동행 상태는 검사하지 않음

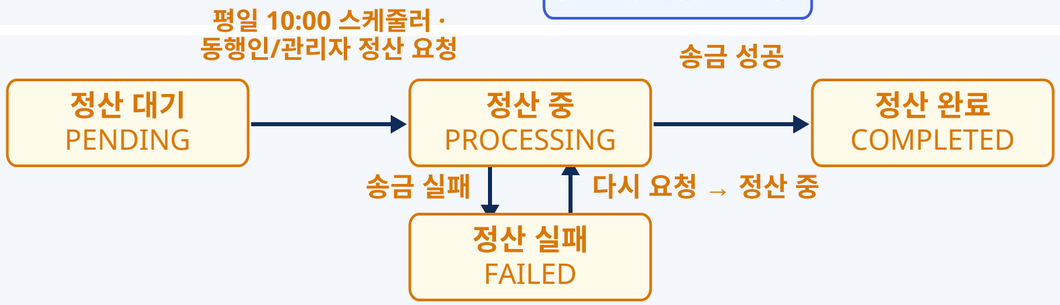


요약은 Claude API로 생성 (작성 요청 안에서 동기 호출)

재시도 스케줄러 없음 → 요약 대기로 남음

정산 Settlement

동행 완료 확인 시 생성
정산일 = 동행 종료 다음 날
노쇼 패널티 있으면 10% 차감



송금 클라이언트는 현재 Mock 구현
정산 완료 후에는 상태 변경 불가

노쇼·등급 NoShow Grade

노쇼 패널티
매칭 후 동행인이 출발 전 취소
→ 지원 NO_SHOW + 패널티 생성



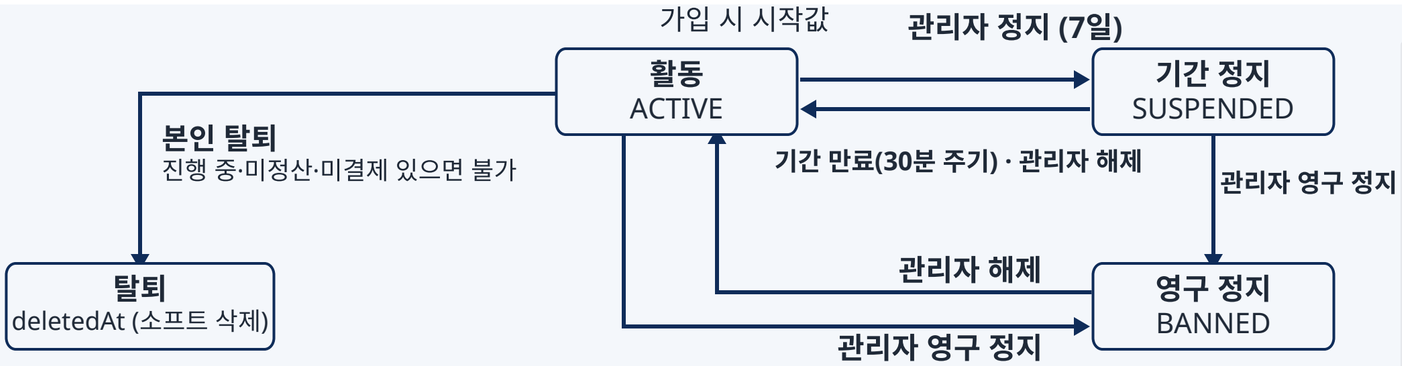
노쇼 횟수 +1 → 등급 즉시 재계산

동행인 등급
동행 완료 횟수로 결정
완료·노쇼 시 즉시 + 매일 03:00 갱신



노쇼가 쌓이면 등급 하락
노쇼 2회 이상 → 한 단계 ↓
노쇼 3회 이상 → 두 단계 ↓

회원 UserStatus



정지 · 영구 정지 · 탈퇴 시 함께 정리 (cancelAllByUser)

의뢰인 → 본인 공고 취소 (모집 중·매칭 후), 결제도 취소

매니저 지원 취소(패널티 없음) · 마감 전 재오픈/후 취소·환불

기간 정지는 30분마다 만료 건을 찾아 활동으로 복구

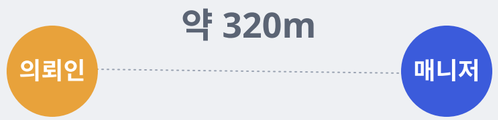
탈퇴는 상태값이 아니라 deletedAt 기록 (소프트 삭제)

그림에 없는 전이는 UserStatus.canTransitionTo에서 차단

실시간 위치 공유와 출발 근접 확인

출발 버튼은 두 사람이 가까워졌을 때만 열린다

멀리 있을 때



거리 100m 초과 — 출발 불가

100m 이내로 가까워졌을 때



반경 100m

거리 약 80m — 출발 가능

서버가 두 위치로 거리를 직접 계산 · **양쪽 위치가 모두 30초 이내**에 갱신된 경우만 인정

동행 단계와 위치 공유 구간

동행 시작 전 › **출발** › 병원 이동 중 › 병원 도착 › 귀가 중 › 귀가 완료 › 위치 삭제

양쪽 위치 실시간 공유 (5초마다 갱신) · 매칭 완료 직후부터 귀가 완료 전까지

거리 조건 (100m·30초)

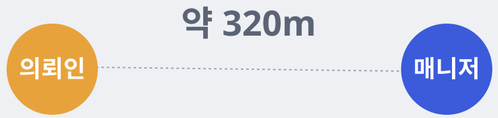
자동 전환 X — 동행 매니저가 버튼을 눌러야 변경

귀가 완료 시 두 위치 삭제
로그에는 좌표 없이 하트비트만

실시간 위치 공유와 출발 근접 확인

출발 버튼은 두 사람이 가까워졌을 때만 열린다

멀리 있을 때



거리 100m 초과 — 출발 불가

100m 이내로 가까워졌을 때



거리 약 80m — 출발 가능

서버가 두 위치로 거리를 직접 계산 · **양쪽 위치가 모두 30초 이내**에 갱신된 경우만 인정

왜 바꿨나

매니저 혼자 출발 처리하면
실제 이용 시간과 결제 금액이 틀어진다

이전
GPS로 집·병원 반경을 판단해
동행 단계를 자동 변경

현재
단계는 매니저 버튼으로만 변경
출발만 가까울 때 허용 (화면+서버 검사)

동행 단계와 위치 공유 구간

동행 시작 전 › **출발** › 병원 이동 중 › 병원 도착 › 귀가 중 › 귀가 완료 › 위치 삭제

양쪽 위치 실시간 공유 (5초마다 갱신) · 매칭 완료 직후부터 귀가 완료 전까지

거리 조건 (100m·30초)

자동 전환 X — 동행 매니저가 버튼을 눌러야 변경

귀가 완료 시 두 위치 삭제
로그에는 좌표 없이 하트비트만

위치로 상태를 바꾸지 않은 이유

위치 반경만으로 상태를 바꾸면

이전 방식(병원 반경 200m 진입 시 병원 도착)에서 검토한 경우

병원 도착
반경 200m 진입

→

진료 중 외출
잠시 병원 밖으로

→

반경 밖 감지
위치가 200m 밖

→

귀가 중?
아직 병원인데

× **실제와 다른 단계로 바뀌면 동행 기록과 시간 기준이 흔들린다**

그래서 바꾼 방식

이전
위치 반경으로 단계 자동 변경
병원 200m · 집 100m

현재
모든 상태를 매니저 버튼으로
출발만 의뢰인과 100m 이내일 때 버튼이 활성화
상태에 영향 X

출발 시각이 결제 금액을 정한다

출발 버튼
의뢰인과 100m 이내
두 위치 모두 30초 이내

→

출발 시각 기록
= 실제 동행 시작 시간
의뢰인과 만나야 기록됨

→

동행 시간
귀가 완료 − 출발
10분 단위 반올림

→

최종 결제 금액
시급 × 동행 시간
차액은 추가 결제·부분 환불

위치로 상태를 바꾸지 않은 이유

위치 반경만으로 상태를 바

이전 방식(병원 반경 200m 진입

병원 도착

반경 200m 진입

× 실제와 다른 단계로 바뀌면

동 변경

튼으로
이내일 때 버튼이 활성화

어르신들의 돌발 행동

병원 동행 후 귀가 중 도착지로 가지 않고 다른 장소로 이동하게 되는 경우

출발 시각이 결제 금액을 정한

출발 버튼

의뢰인과 100m 이내
두 위치 모두 30초 이내

→

출발 시각 기록

= 실제 동행 시작 시간
의뢰인과 만나야 기록됨

→

동행 시간

귀가 완료 − 출발
10분 단위 반올림

→

최종 결제 금액

시급 × 동행 시간
차액은 추가 결제·부분 환불

위치로 상태를 바꾸지 않은 이유

위치 반경만으로 상태를 바

이전 방식(병원 반경 200m 진입

병원 도착

반경 200m 진입

× 실제와 다른 단계로 바뀌면

동 변경

튼으로
이내일 때 버튼이 활성화

어르신들의 돌발 행동

병원 동행 후 귀가 중 도착지로 가지 않고 다른 장소로 이동하게 되는 경우

병원 도착 전 다른 병원으로 진료

출발 시각이 결제 금액을 정한

출발 버튼
의뢰인과 100m 이내
두 위치 모두 30초 이내

→

출발 시각 기록
= 실제 동행 시작 시간
의뢰인과 만나야 기록됨

→

동행 시간
귀가 완료 – 출발
10분 단위 반올림

→

최종 결제 금액
시급 × 동행 시간
차액은 추가 결제·부분 환불

탈퇴·정지 시점별 정리 · 진행 중 동행은 제외

	공고 등록	모집 마감	출발	귀가 완료
	모집 기간 · OPEN / MATCHED	마감 후 · MATCHED / EXPIRED	동행 진행 중 · IN_PROGRESS	
의뢰인 탈퇴 · 정지 cancelAllByClient	**공고 → CANCELED** 지원 전부 취소 (패널티 없음, 동행 매니저 알림) · 결제 취소·환불 모집 마감 전후 같은 처리		**정리하지 않음** 탈퇴는 거절 정지는 진행 중 건 제외	
매니저 탈퇴 · 정지 cancelAllByEscort	**지원 취소 · 공고 재오픈** 대기 지원 → 취소 (패널티 없음) 승인 공고 → OPEN 다시 모집 · 환불 없음	**공고 취소 · 환불** 승인 공고 → CANCELED 결제 환불 · 의뢰인 알림	**정리하지 않음** 탈퇴는 거절 정지는 진행 중 건 제외	

탈퇴: 진행 중 동행 · 의뢰인 추가 결제 미결제 · 매니저 미정산이 있으면 거절
정지: 바로 적용

개인정보 암호화와 해시

원문 (개인정보)	알고리즘	같은 입력을 두 번 넣으면?	속도	원문을 다시 볼 수 있나?
신원 정보, 비상연락처, 계좌 정보, 특이사항, 채팅 내용	**AES-256-GCM** 대칭키 암호화	홍길동 → a8F3…k2 홍길동 → Zq7P…9x ≠ 다른 결과	빠름	✓ **복호화 가능** 암호문 → 복호화 → 원문 표시
이메일, 전화번호 암호화본도 함께 저장	**HMAC-SHA256** 키를 쓰는 해시	홍길동 → 9c1e…b4 홍길동 → 9c1e…b4 = 같은 결과	빠름	× **복호화 불가** 같은 값 찾기 · 유니크 제약에 사용
비밀번호	**BCrypt** 비밀번호용 해시	pw1234 → $2a…Xq1 pw1234 → $2a…m7P ≠ 다른 결과	일부러 느림	× **복호화 불가** 로그인 때 입력값과 비교만 함

DB가 유출돼도 원문은 그대로 보이지 않는다

실시간 알림 추가

흐름

① 결제·지원 등 본 작업
send() → 이벤트 발행

➡

② 커밋 후 · @Async
알림 DB 저장

➡

③ 저장 직후 이벤트
SSE로 실시간 전송

➡

④ 받는 사람 화면
알림 → 누르면 해당 화면

알림 17종

동행

동행·보고서·리뷰 9종

지원·승인·거절·취소, 진행 단계 변경
보고서·리뷰 작성 요청과 작성 완료

결제

결제·정산 5종

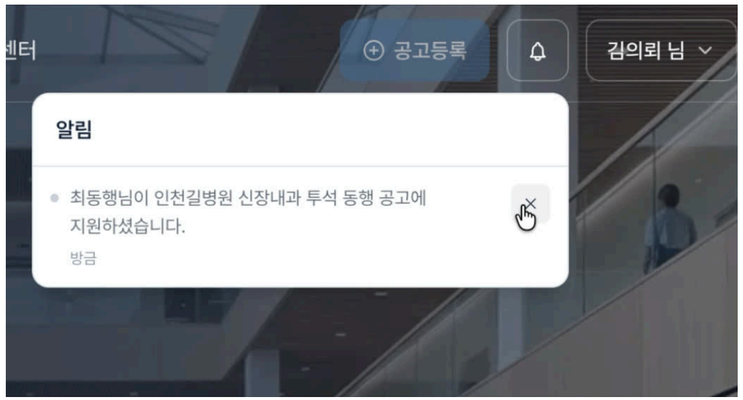
결제 완료·취소·부분 환불 (금액 포함)
추가 결제 요청 · 정산 완료

공고·회원

공고 삭제, 회원 탈퇴·정지 3종

공고 삭제 → 의뢰인·지원한 매니저
회원 탈퇴·정지 → 지원·공고의 상대

```
// PaymentService
noticeService.send(
  userId,
  C_PAYMENT_DONE.of(payment.post.title, payment.amount.toString()),
  U_PAYMENT_DONE.of(payment.post.id!!))
// NotificationEventListener
emitterRepository.findAllByUserId(event.receiverId).forEach {
  (key, emitter) -> sendToClient(emitter, key, ...)}
```



1:1 실시간 채팅 추가 · 공고에서 바로 연락한다

공고 상세 '연락하기'로 상대와 바로 대화

1:1

의뢰인 ↔ 매니저, 같은 역할끼리도 가능
같은 상대와는 방 1개 · 있으면 다시 연다

연결 /ws/chat · JWT로 사용자 확인

구독 /topic/chat/{방} · 방 참여자만 허용

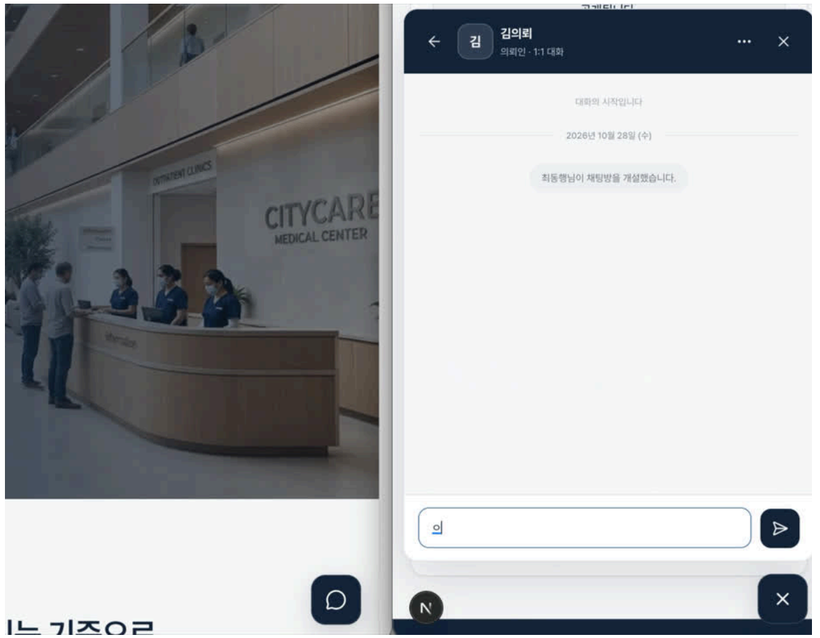
보내기 → **DB 저장·암호화** → **커밋 후 전송** → 같은 방 구독자 화면에 바로 표시

채팅 기능

1 안 읽은 수
채팅방 목록에 방마다 표시
읽은 시각 이후 상대 메시지만

2 나가기
상대가 보내면 다시 입장

3 암호화
메시지를 AES-256-GCM으로 저장

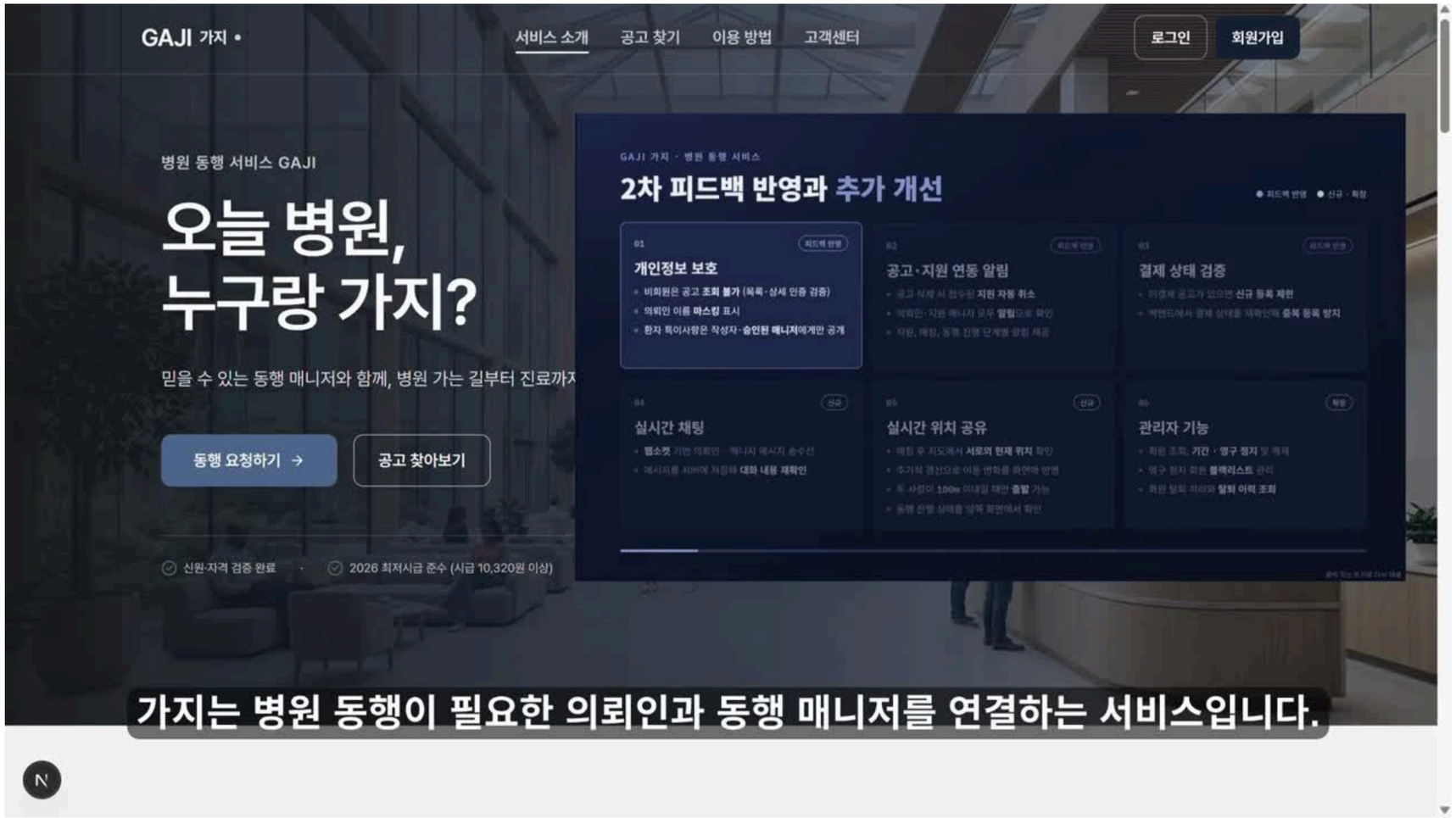


05

시연과 상태 다이어그램

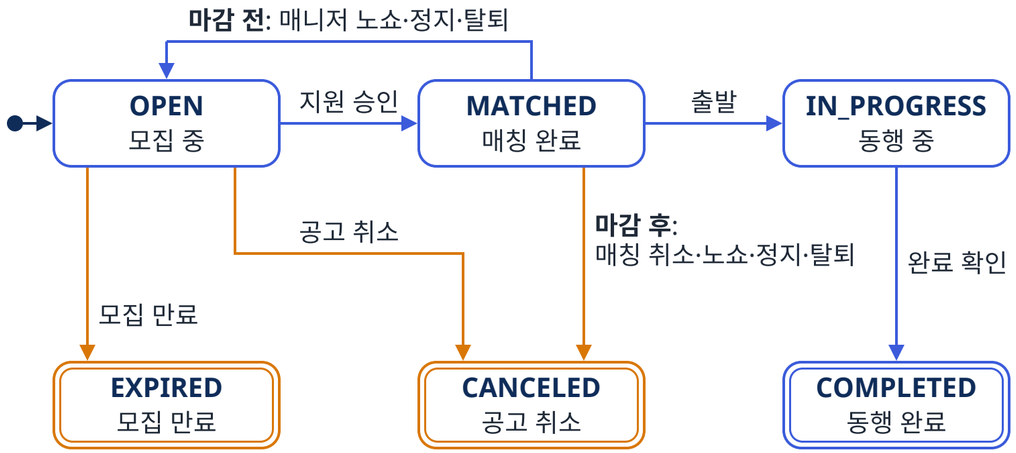
실제 화면과 상태 다이어그램

서비스 시연

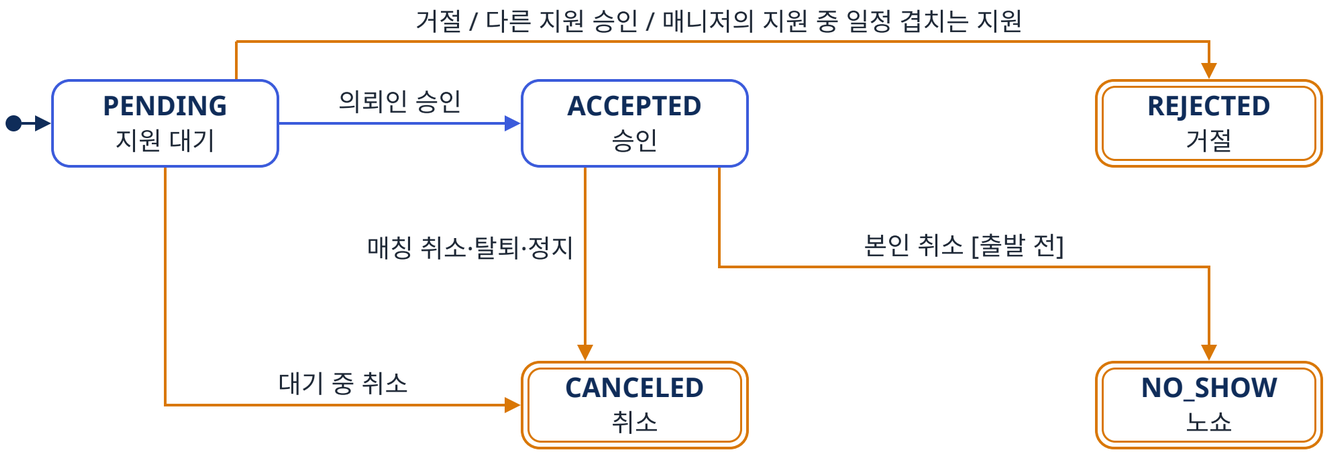

GAJI 가지 •
서비스 소개
공고 찾기
이용 방법
고객센터
로그인
회원가입
병원 동행 서비스 GAJI
오늘 병원,
누구랑 가지?
믿을 수 있는 동행 매니저와 함께, 병원 가는 길부터 진료까지
동행 요청하기 →
공고 찾아보기
신원·자격 검증 완료
2026 최저시급 준수 (시급 10,320원 이상)
GAJI 가지 · 병원 동행 서비스
2차 피드백 반영과 추가 개선
개인정보 보호
비회원은 공고 조회 불가 (목록·상세 인증 검증)
의뢰인 이름 마스킹 표시
환자 특이사항은 작성자·승인된 매니저에게만 공개
공고·지원 연동 알림
결제 상태 검증
실시간 채팅
실시간 위치 공유
관리자 기능
가지는 병원 동행이 필요한 의뢰인과 동행 매니저를 연결하는 서비스입니다.

상태 전이 다이어그램 ①

공고 PostStatus

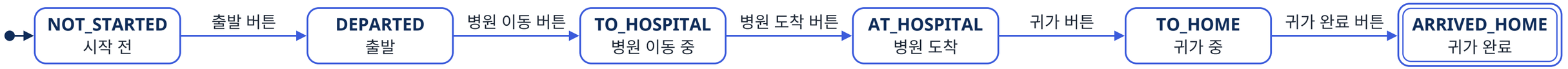


지원 ApplicationStatus



동행 완료 뒤에도 지원은 ACCEPTED / 취소 후 재지원은 새 Application 생성

동행 진행 EscortProgress



최신 진행 로그가 없으면 NOT_STARTED

출발: 의뢰인·매니저 100m 이내 + 위치 전송 30초 이내 (서버 검사)

귀가 완료 시 위치 삭제

이동 RideStatus



병원행: TO_HOSPITAL → AT_HOSPITAL

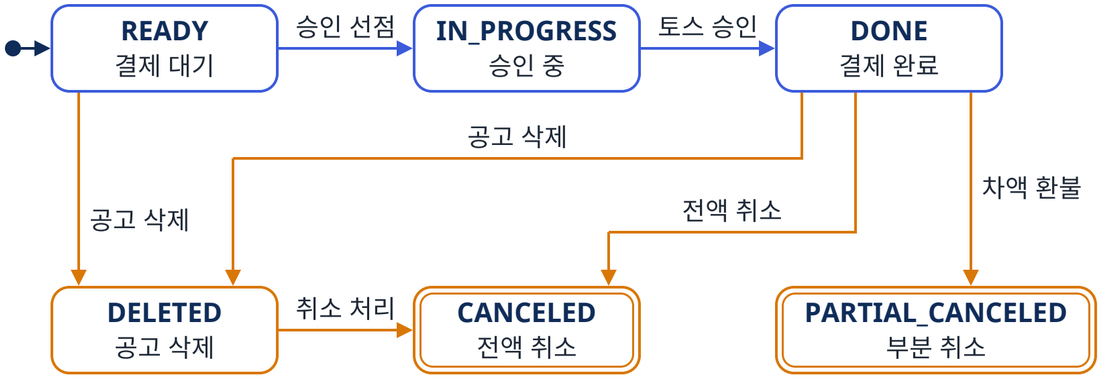
귀가행: TO_HOME → ARRIVED_HOME

공고 등록 시 두 Ride 생성 / 두 방향을 독립적으로 관리

시작 화살표 = 생성/초기값 이중 테두리 = 후속 전이 없음 주황 테두리 = 예외·취소 상태 실선 = 허용 전이 별도 생성·삭제·계산값은 설명으로 구분

상태 전이 다이어그램 ②

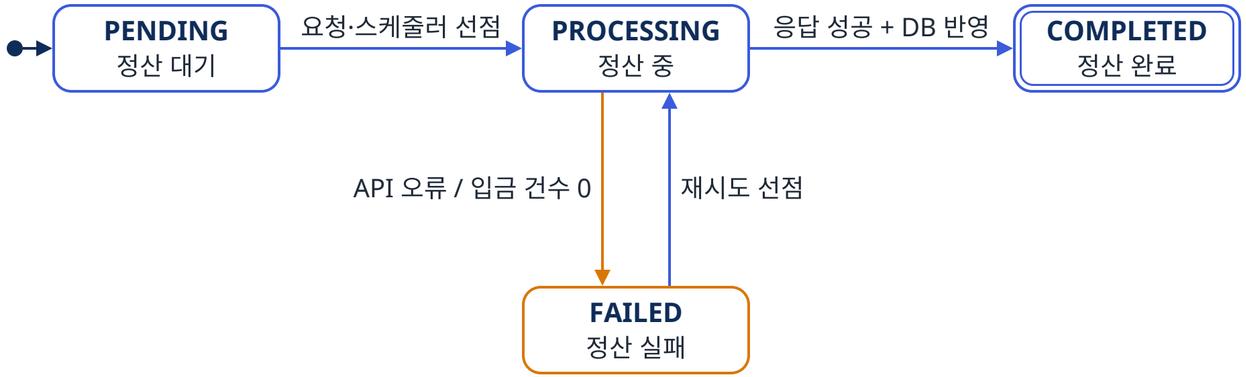
결제 PaymentStatus



승인 오류: 결제 행 삭제 / 사용자 전액 취소: 별도 READY 결제 생성

PARTIAL_CANCELED는 enum상 종료 상태: 이후 취소를 허용하는 서비스 조건과 불일치

정산 SettlementStatus



응답 성공 뒤 DB 실패: PROCESSING 유지 / 현재 외부 구현은 MockSettlementClient

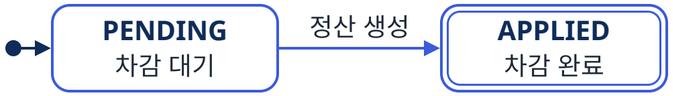
생성: 공고 완료 확인 시 / 완료 뒤 정산 상태 변경 불가

회원 UserStatus

기간 정지: 7일 기준 / 만료는 30분 주기로 확인



노쇼 패널티



노쇼 때 생성 → 다음 정산 생성 시 1건 적용

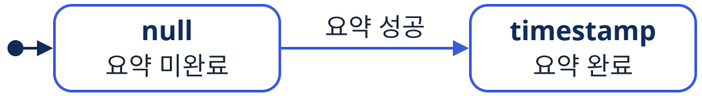
수수료 제외 정산금의 10% 차감

탈퇴는 UserStatus가 아님: deletedAt 기록 + 공고·지원 취소 등 처리

등급은 계산값: SEED / SPROUT / FLOWER / EGGPLANT

완료 0·5·10·30회 기준 / 노쇼 2회면 1단계, 3회 이상이면 2단계 하향

보고서 요약 (별도 enum 없음)



summarizedAt으로 구분 / 실패해도 null 유지

원문 저장 후 동기 호출 / 자동 재시도 없음

시작 화살표 = 생성/초기값　이중 테두리 = 후속 전이 없음　주황 테두리 = 예외·취소 상태　실선 = 허용 전이　별도 생성·삭제·계산값은 설명으로 구분

상태 간 진행 조건 ①

[조건]을 모두 만족할 때만 오른쪽으로 진행 / 미충족이면 요청 거절

공고 등록	[회원 ACTIVE(활성), 역할 CLIENT(매니저) / ADMIN(관리자)] [결제 READY(대기) 0건]	**새 공고 OPEN** **결제 READY**
교육 이수	[매니저 프로필 존재] [필수 교육 영상 전부 completed(완료여부) = true]	**프로필** **verified(검증여부) = true**
공고 지원	[공고 OPEN(모집), 모집 시작 ≤ 현재 ≤ 모집 끝] [매니저 ACTIVE(활성), 교육 이수 완료, 같은 공고 재지원은 기존 지원이 CANCELED(취소) 상태만 허용]	**새 지원** **PENDING(대기)**
지원 승인	[본인 공고 OPEN(모집), 지원 PENDING(대기), 승인할 매니저 ACTIVE(활성)] [성공 결제 존재, 해당 매니저의 ACCEPTED(승인) 일정과 중복 없음]	**지원 ACCEPTED(승인)** **공고 MATCHED(매칭)**

상태 간 진행 조건 ②

서비스와 조건부 UPDATE가 실제로 검사하는 조건을 구분

버튼 진행

[동행 매니저 본인, 지원 상태 ACCEPTED(승인)]
[최신 진행 로그의 바로 다음 단계만 / 출발은 의뢰인 100m·30초 이내]

→ **다음 진행 로그 저장**

DEPARTED(출발)는 공고 MATCHED(매칭) → IN_PROGRESS(동행 중)도 수행. 나머지 버튼 단계는 공고·회원 상태를 직접 재검사하지 않음.

위치 공유

[지원 ACCEPTED(승인), 공고 MATCHED(매칭) / IN_PROGRESS(동행 중)]
[ARRIVED_HOME(귀가 완료) 로그 없음: DB 조건부 UPDATE에서도 재검사]

→ **최근 위치 저장 거리·nearby 계산**

위치로 단계가 바뀌지 않는다. 두 위치가 100m·30초 이내면 nearby = true → 출발 버튼 활성화, 출발 시 서버가 다시 확인(400-37).

완료 확인

[의뢰인: 본인 공고 IN_PROGRESS(동행 중) / 관리자는 이 검사 생략]
[DEPARTED(출발)·ARRIVED_HOME(귀가 완료) 기록, 매니저 프로필 존재]

→ **공고 COMPLETED(완료) 정산 PENDING(대기)**

귀가 완료만으로 공고가 완료되지는 않는다. 실제 시간으로 차액 환불·추가 READY(대기) 결제 생성, 정산은 함께 생성.

리뷰 작성

[공고 작성 의뢰인 본인]
[지원 ACCEPTED(승인), 공고 COMPLETED(완료), 해당 지원의 기존 리뷰 없음]

→ **리뷰 저장 평점 반영**

근접 기준: 100m · 30초. 귀가 완료·완료·취소 시 두 사람 위치 삭제, 이후 저장·조회 차단.

상태 간 진행 조건 ③

업무 취소와 회원 정지의 후속 처리, 그리고 구현상 검사하지 않는 조건을 분리한다

지원 취소

[본인 지원 PENDING(대기) / ACCEPTED(승인)]
[공고 IN_PROGRESS(진행 중)·COMPLETED(완료)는 불가]

→ **대기: CANCELED(취소)**
승인: NO_SHOW

노쇼는 모집 끝 전이면 공고 OPEN(모집), 끝 이후면 CANCELED(취소)·환불. 패널티 생성과 노쇼 횟수 증가.

결제 취소

[사용자 직접 취소: 본인 결제, 공고 OPEN(모집) / EXPIRED(만료)]
[결제 DONE(완료) / PARTIAL_CANCELED(부분 취소), canceledAt 미선점, 결제키 존재]

→ **결제 취소**
새 결제 READY(대기) 생성

정산 요청

[본인 정산 + 계좌 프로필 / 관리자는 소유자 검사 생략]
[PENDING(대기) / FAILED(실패)에서만 PROCESSING(진행 중) 선점 성공]

→ **PROCESSING**
외부 API 요청

스케줄러만 settledDate ≤ 오늘 검사. 직접 요청에는 날짜 검사 X

회원 탈퇴

[의뢰인: IN_PROGRESS(동행 중) 공고 없음, 미결제 공고 없음]
[매니저: ACCEPTED(승인) + IN_PROGRESS(동행 중) 없음, 미완료 정산 없음]

→ **deletedAt 기록**
토큰·프로필 정리

정지·탈퇴 시 지원은 패널티 없이 취소. 매니저가 승인된 지원의 공고가 마감 전 재오픈, 마감 후 취소·환불.

06

인프라와 회고

인프라·성능·남은 과제

인프라 이전

2차 · Railway 싱가포르

Vercel
프론트

모니터링 없음

Railway - WAS
1 vCPU · RAM 0.5GB · 8GB

도커 MySQL 컨테이너

3차 · 서울 ap-northeast-2

Vercel

EC2 · Docker
t3.medium · 2 vCPU · 4GB · 8GB

Prometheus · Grafana
모니터링 별도 EC2 Instance

Supabase PostgreSQL
DB 별도 클라우드 서비스

비용



지역

싱가포르
Railway

서울
ap-northeast-2

확장성



스케일 아웃

DB 분리



WAS 부담 감소

k6를 활용한 공고 목록 성능 개선

원인 1 인덱스 없는 전체 스캔 2 캐시가 빌 때 요청이 몰림 → 커넥션 풀 포화, 느린 1%가 1초 가까이 대기

방안 선택 · PR #94 머지

선택	**결제 인덱스**	전체 스캔 해결 · 쿼리 **102 → 5ms**
선택	**캐시 sync**	동시에 몰리는 조회를 한 번으로
제외	**풀 크기 늘리기**	원인은 그대로
제외	**상태·마감일 인덱스**	지금 규모에선 쓰이지 않음
제외	**N+1 제거**	이미 적용되어 있음

결과 (수정 전 → 후)

	수정 전	수정 후
p95 응답 시간	126ms	18.5ms 약 6.8배
요청당 커넥션 점유	176ms	17.7ms 약 10배
평균 응답 시간	30.4ms	8.5ms 약 3.6배

풀 대기 최대 0건 (3회 측정 모두)

한계점

- 로컬 · 공고 약 1,100건 기준
- 느린 소수 요청(캐시 미스)이 좌우
- 위치·로그인 API는 후속 과제

측정: k6 · 50명 · 30초 · 단계마다 서버 재시작 · 워밍업 제외 3회 중앙값

약속한 것과 추가한 것

4/5 2차 약속 이행 반영

1 부분 반영 삭제 상태값 · deletedAt

5 추가 기능 이유와 장 번호

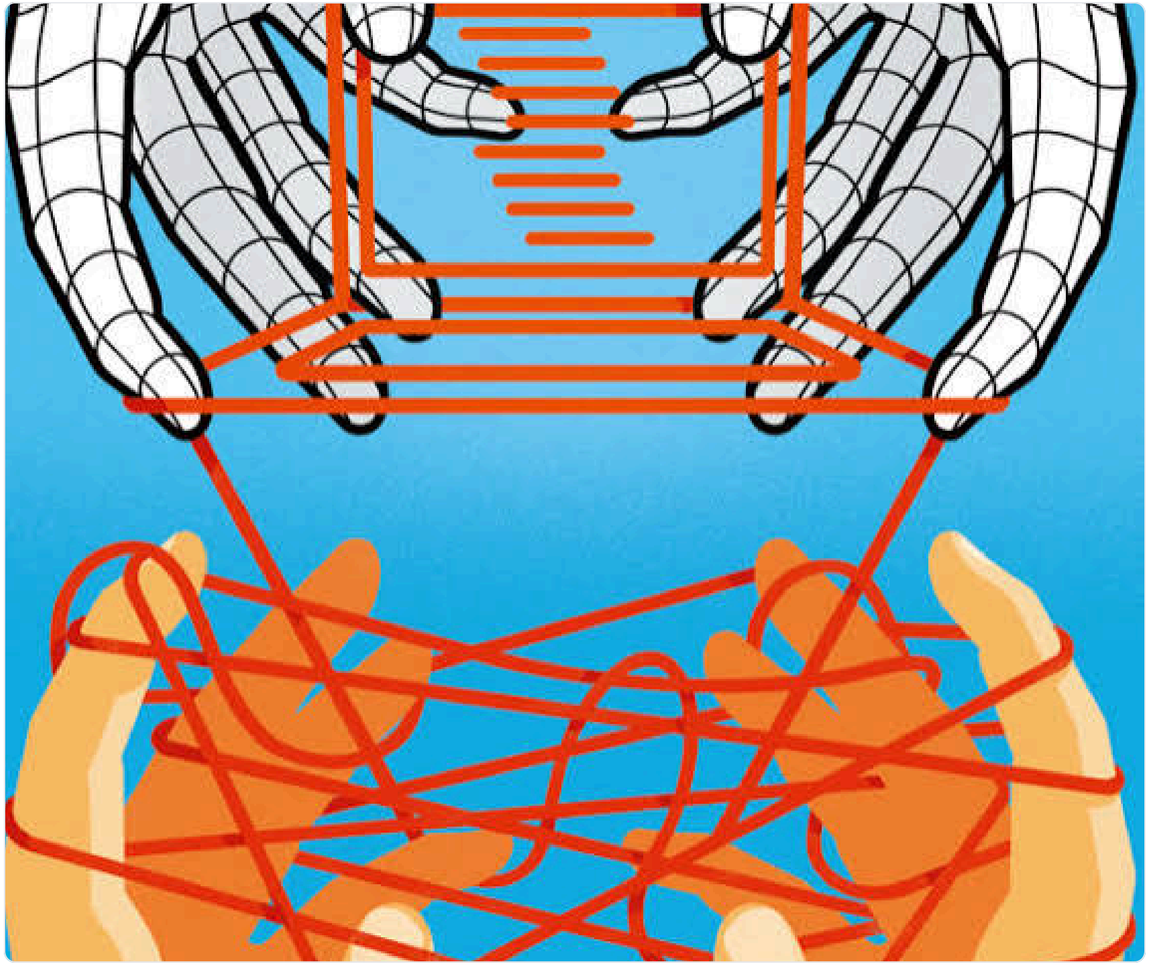
2차 약속 이행

상태·동시성	✓ 반영
결제 서버 검증	✓ 반영
AI 설정 yaml 분리	✓ 반영
삭제 상태값	◐ 부분
성능 테스트	✓ 반영

추가 기능의 이유

실좌표 위치 공유	이용 시간·금액 보호	19장
정지·탈퇴 정리	잔여 업무 처리	21장
개인정보 암호화	개인정보 유출 예방	22장
알림	사용자 경험(UX) 개선	23장
채팅	사용자 경험(UX) 개선	24장

회고



최훈희 팀장

여러 상태가 맞물린 서비스를 개발하며, 상태 전이를 엔티티 안에서 일관되게 관리하는 방법을 익혔습니다. 또한 팀원들과 설계를 공유하고 리뷰를 주고받으며 협업과 커뮤니케이션 역량을 키울 수 있었습니다.

김신영

코틀린 마이그레이션뿐만 아니라 서버 검증과 규칙 통합을 하고 캐시와 인덱스를 k6 측정으로 최적화방식을 익히고 동료들과 협업하면서 함께 문제를 해결해 나가면서 협업경험도 기를 수 있었습니다.

나희원

기존 프로젝트의 문제점을 발견하고 개선하는 과정에서 기능 구현뿐만이 아니라 안정성과 유지보수성도 중요하다는 것을 배웠습니다. 또한 코틀린 전환하면서 기존 로직을 더 깊게 이해할 수 있었습니다.

박현호

개발자가 편하게 개발을 할 수 있도록 다양한 라이브러리가 있다는 것을 느끼게 되었고, 코틀린이라는 매력적인 언어를 경험해보며 컴파일에서 에러를 잡을 수 있다는 부분이 정말 마음에 들었습니다.

감사합니다.

오늘 병원, 누구랑 가지?

Team06 · 훈희는 못말려

https://www.devcproject.cloud

https://github.com/prgrms-be-devcourse/NBE-12-14-3-Team06